

Re-understanding Graph Unlearning through Memorization

Pengfei Ding
Macquarie University
Sydney, Australia
pengfei.ding@mq.edu.au

Yan Wang*
Macquarie University
Sydney, Australia
yan.wang@mq.edu.au

Guanfeng Liu
Macquarie University
Sydney, Australia
guanfeng.liu@mq.edu.au

Abstract

Graph unlearning (GU), which removes nodes, edges, or features from trained graph neural networks (GNNs), is crucial in Web applications where graph data may contain sensitive, mislabeled, or malicious information. However, existing GU methods lack a clear understanding of the key factors that determine unlearning effectiveness, leading to three fundamental limitations: (1) impractical and inaccurate GU difficulty assessment due to test-access requirements and invalid assumptions, (2) ineffectiveness on hard-to-unlearn tasks, and (3) misaligned evaluation protocols that overemphasize easy tasks and fail to capture true forgetting capability. To address these issues, we establish GNN memorization as a new perspective for understanding graph unlearning and propose MGU, a Memorization-guided Graph Unlearning framework. MGU achieves three key advances: it provides accurate and practical difficulty assessment across different GU tasks, develops an adaptive strategy that dynamically adjusts unlearning objectives based on difficulty levels, and establishes a comprehensive evaluation protocol that aligns with practical requirements. Extensive experiments on ten real-world graphs demonstrate that MGU consistently outperforms state-of-the-art baselines in forgetting quality, computational efficiency, and utility preservation.

CCS Concepts

• Security and privacy → Privacy-preserving protocols.

Keywords

Graph Unlearning; Machine Unlearning; Graph Neural Networks

ACM Reference Format:

Pengfei Ding, Yan Wang, and Guanfeng Liu. 2026. Re-understanding Graph Unlearning through Memorization. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792383>

1 Introduction

The widespread use of Graph Neural Networks (GNNs) in Web-related domains (e.g., social networks [14], recommender systems [25], financial applications [33]) poses unprecedented opportunities and challenges. In the Web ecosystem, graph-structured data inevitably includes sensitive, erroneous, or malicious information (e.g., privacy-sensitive records, mislabeled interactions, fraudulent

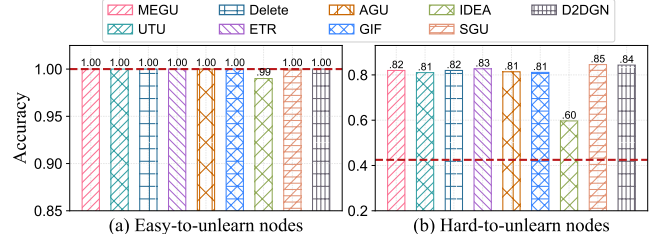


Figure 1: Unlearning performance on the 10% easiest and 10% hardest nodes in the Cora dataset. The red dashed line denotes the gold-standard performance obtained by retraining.

transactions [32]), which not only degrades the reliability of Web services but also raises critical privacy and security concerns, highlighting the necessity of removing such information from trained GNNs. Graph Unlearning (GU) [10, 12] has thus emerged as a crucial paradigm for the Web, enabling efficient removal of specific graph elements (nodes, edges, or features) without costly full retraining.

Despite growing research interest, the key factors that determine graph unlearning effectiveness remain unclear. Understanding what makes an unlearning task easy or hard is crucial, as such insights can guide the development of unlearning methods that are robust against challenging cases, and support evaluation protocols aligned with practical requirements [45]. Motivated by this, we investigate whether existing methods can effectively assess graph unlearning difficulty. However, our study shows that current difficulty assessments are both impractical and inaccurate, which not only masks the ineffectiveness of existing methods on hard-to-unlearn tasks, but also leads to misaligned evaluation protocols. Specifically, we identify the following three interconnected limitations:

Limitation 1: Impractical and inaccurate difficulty assessment. Existing assessment strategies [5, 6] suffer from two key issues: (1) they require access to test data or already-unlearned models, which are typically unavailable in practice; and (2) they assume that unlearning difficulty depends on graph connectivity distance to the test set. However, our analysis reveals that unlearning difficulty shows no correlation with test-set distance (see Section 4.2).

Limitation 2: Ineffectiveness on hard-to-unlearn tasks. Our difficulty assessment reveals a critical blind spot: existing GU methods perform well on easy-to-unlearn tasks but fail dramatically on hard ones. Figure 1 illustrates this gap on the Cora dataset with a GCN backbone [19], where we compare unlearning performance on the 10% easiest and 10% hardest training nodes (ranked by our metric) and evaluate forgetting quality by re-inputting deleted nodes into the unlearned model. Ideally, effective unlearning should yield predictions consistent with the *gold standard*, i.e., a model retrained from scratch without the deleted nodes (red dashed line in Figure 1). While all methods achieve near-optimal results on easy-to-unlearn

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792383>

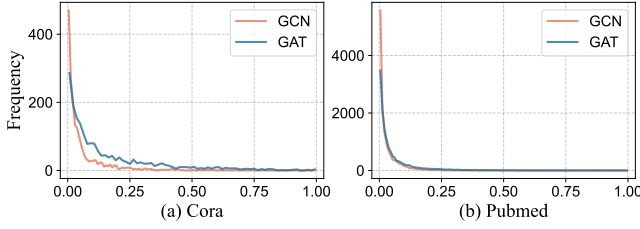


Figure 2: Long-tailed distribution of node unlearning difficulty (via Eq. (2)). Higher scores indicate harder cases.

nodes, their performance drops by over 40% on hard ones, as unlearned models continue predicting the deleted nodes’ labels with high accuracy, rather than exhibiting the retrained model’s uncertainty. This degradation consistently appears across multiple datasets, GNN backbones, and unlearning tasks (see Appendix A), indicating a systematic limitation of existing GU methods.

Limitation 3: Misaligned evaluation protocols. The above limitations reveal a misalignment between current evaluation protocols and practical needs in two key aspects: (1) **Selection bias:** Most evaluations select unlearning targets randomly [10, 36], which fails to reflect realistic unlearning scenarios. As shown in Figure 2, unlearning difficulty follows a long-tailed distribution, where random sampling tends to favor easy-to-unlearn elements and overlooks hard ones. However, these hard-to-unlearn elements, though rare and structurally irregular, often correspond to actual unlearning targets in practice (e.g., privacy-sensitive outliers, mislabeled data, malicious accounts [13, 45]). (2) **Insufficient verification:** Current evaluations focus on utility preservation but rarely verify whether deleted information is fully removed [22, 39]. Some methods assess forgetting on deleted elements [12] but assume uniform degradation in predictions (e.g., convergence to random guessing), ignoring that elements at different difficulty levels exhibit distinct forgetting behaviors. As shown in Figure 1, gold-standard retraining produces different patterns for easy and hard tasks, illustrating how this uniform assumption can mislead the design of GU methods.

These limitations reveal that existing studies understand GU from superficial perspectives, emphasizing structural effects or empirical observations while overlooking the fundamental insight: *the way a model memorizes information inherently determines how it should be forgotten*. Memorization characterizes how a model encodes and retains information during training, fundamentally governing the persistence of learned knowledge and the difficulty of removing it [13, 35]. By exploring how GNNs memorize graph elements to varying degrees, we can better understand why some elements are harder to unlearn and how to handle them effectively.

Our Work. In this paper, we establish GNN memorization as a new perspective for understanding GU, and propose MGU, a **Memorization-guided Graph Unlearning** framework. To address **Limitation 1**, we establish the relationship between memorization and unlearning difficulty, and design an accurate assessment framework without test-access requirement. To address **Limitation 2**, we propose an adaptive unlearning strategy that adjusts objectives based on how unlearning difficulty affects generalization. To address **Limitation 3**, we propose a novel evaluation protocol

with difficulty-aware sampling and comprehensive performance assessment that more faithfully reflects real-world requirements.

Our main contributions are summarized as follows:

- **Novel Perspective:** We establish the first connection between GNN memorization and unlearning difficulty, providing new insights into why existing methods fail on hard-to-unlearn tasks.
- **Universal Adaptive Framework:** We propose a general model-agnostic unlearning framework that adaptively handles both easy and hard GU tasks without accessing model’s internal structure.
- **Comprehensive Evaluation Protocol:** We design a difficulty-aware sampling strategy with comprehensive performance assessment that better aligns with real-world requirements.
- **SOTA Performance:** Extensive experiments on ten real-world graphs demonstrate the superiority of MGU in forgetting quality, computational efficiency, and utility preservation.

2 Related Work

2.1 Graph Unlearning

Early research on GU primarily focused on specific tasks or GNN architectures: (1) Partition-based methods [4, 32, 43] partition the graph into shards and retrain only the affected submodels to reduce computational cost. However, these methods are restricted to node unlearning and become inefficient on large-scale graphs. (2) Closed-form methods [7, 8, 40] enable retraining-free updates with theoretical guarantees, but rely on stringent assumptions about linear GNNs and lack generalizability.

To overcome these limitations, recent studies [20, 39] propose general frameworks for diverse GU tasks and GNN backbones. (1) Influence function-based methods [11, 36] approximate parameter updates for efficient unlearning across tasks. (2) Learning-based methods [6, 10, 22] design specialized loss functions to balance forgetting and reasoning. However, these methods remain effective on easy-to-unlearn tasks and degrade significantly on hard ones.

2.2 Memorization in Deep Neural Networks

In Euclidean domains such as images and text, memorization in deep neural networks (DNNs) has been extensively studied [1, 35]. Recent theories [41] show that memorization is essential for near-optimal generalization. Empirical studies [45] further reveal that atypical or noisy samples are more likely to be memorized.

Despite extensive progress in Euclidean domains, studies of memorization in GNNs remain limited, and its connection to unlearning difficulty is unexplored. A recent work [18] extended memorization analysis to GNNs but ignored graph connectivity, focusing only on nodes without considering other graph elements (edges and features). In this work, we present the first systematic study of GNN memorization and reveal its key implications for graph unlearning.

3 Preliminary

3.1 Notations and Background

In general, a graph is represented as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{X})$, where $\mathcal{V}$ is the set of nodes, $\mathcal{E}$ is the set of edges, and $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ denotes the node feature matrix. Each node $v_i \in \mathcal{V}$ is associated with a d -dimensional feature vector $\mathbf{x}_i \in \mathbb{R}^d$. In this paper, we focus on the commonly studied node classification task on $\mathcal{G}$. Given labels

for a subset of training nodes $\mathcal{D}_{\text{train}} \subset \mathcal{V}$, denoted as $\mathcal{Y}_{\text{train}} = \{y_1, y_2, \dots, y_m\}$, where each $y_i \in \{1, \dots, |C|\}$ and C is the set of all classes. The objective is to learn model parameters θ such that the trained model f_θ can accurately predict the labels of nodes in the test set $\mathcal{D}_{\text{test}}$, where $\mathcal{D}_{\text{test}} \cap \mathcal{D}_{\text{train}} = \emptyset$.

3.2 Graph Unlearning Formalization

Given an original GNN model f_θ trained on a graph $\mathcal{G}$ and an unlearning request $\Delta\mathcal{G}$, the goal of graph unlearning is to obtain an *unlearned* model f_{θ^u} that closely approximates the *retrained* model f_{θ^r} , where f_{θ^r} is retrained from scratch on the remaining graph $\mathcal{G} \setminus \Delta\mathcal{G}$. Graph unlearning can be categorized into three tasks: (1) *Node Unlearning* ($\Delta\mathcal{G} = \{\Delta\mathcal{V}, \emptyset, \emptyset\}$), where a set of nodes (and their associated information) is removed; (2) *Edge Unlearning* ($\Delta\mathcal{G} = \{\emptyset, \Delta\mathcal{E}, \emptyset\}$), where a set of edges is removed; and (3) *Feature Unlearning* ($\Delta\mathcal{G} = \{\emptyset, \emptyset, \Delta\mathcal{X}\}$), where a set of node features is removed, typically masked or replaced with zeros.

3.3 General Memorization Mechanism

Memorization was first introduced in computer vision [13] to quantify a model’s reliance on specific training samples for prediction. Given a training dataset $\mathcal{D}_{\text{train}}$, training algorithm $\mathcal{T}$, and sample $i \in \mathcal{D}_{\text{train}}$ with feature $\mathbf{x}_i$ and label y_i , the memorization score is:

$$\text{mem}(i) = \Pr_{f \sim \mathcal{T}(\mathcal{D}_{\text{train}})} [f(\mathbf{x}_i) = y_i] - \Pr_{f \sim \mathcal{T}(\mathcal{D}_{\text{train}} \setminus i)} [f(\mathbf{x}_i) = y_i], \quad (1)$$

where the first term measures the prediction probability with sample i included in training, while the second measures it when i is excluded. A higher score indicates stronger memorization. Although well studied in Euclidean domains, extending this mechanism to graphs requires considering the impact of graph connectivity.

4 Methodology

4.1 Framework Overview

MGU consists of three main components: (1) *Memorization-based difficulty assessment* leverages memorization strength to estimate unlearning difficulty; (2) *Adaptive unlearning strategy* defines adaptive unlearning objectives based on task difficulty and generalization impact; (3) *Comprehensive evaluation protocol* employs difficulty-aware sampling and task-aware metrics to assess the robustness, practicality, and generalizability of GU methods.

4.2 Memorization-based Difficulty Assessment

Memorization Evaluation on Graphs. We first quantify the memorization degree of graph elements in GNNs. The memorization score in Eq. (1) originates from Euclidean domains, where training samples are assumed to be independent. However, this assumption fails in graphs, where a node’s prediction depends not only on its own features but also on information propagated from its neighbors. Therefore, memorization evaluation on graphs must consider both the direct effect on the target node and the indirect influence on its neighbors. To capture these dual effects, we extend Eq. (1) by incorporating a neighbor-impact term:

$$\text{mem}(v_i) = \alpha \cdot |\Delta(v_i)| + (1 - \alpha) \cdot \Delta\mathcal{N}_k(v_i), \quad (2)$$

where $\Delta(v_i)$ is defined following Eq. (1) to measure the change in prediction probability when v_i is excluded from $\mathcal{D}_{\text{train}}$. $\Delta\mathcal{N}_k(v_i)$

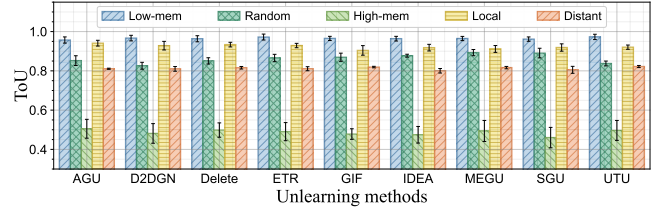


Figure 3: ToU performance of GU methods for node unlearning on Cora with GAT backbone ($N = 5$).

aggregates the impact on its k -hop neighbors (k equals the number of GNN layers). $\alpha \in [0, 1]$ balances the two terms, and absolute values capture their magnitudes. Specifically, $\Delta\mathcal{N}_k(v_i)$ is computed as the weighted sum of prediction changes across neighbors:

$$\Delta\mathcal{N}_k(v_i) = \sum_{v_j \in \mathcal{N}_k(v_i)} w_{ij} \left(\Pr_{f \sim \mathcal{T}(\mathcal{D}_{\text{train}})} [f(\mathbf{x}_j) = y_j] - \Pr_{f \sim \mathcal{T}(\mathcal{D}_{\text{train}} \setminus v_i)} [f(\mathbf{x}_j) = y_j] \right), \quad (3)$$

where the weight w_{ij} decays exponentially with graph distance:

$$w_{ij} = \frac{\beta^{\text{dis}(v_i, v_j)}}{\sum_{v_t \in \mathcal{N}_k(v_i)} \beta^{\text{dis}(v_i, v_t)}}, \quad (4)$$

where $\text{dis}(v_i, v_j)$ denotes the shortest path length between v_i and v_j , and $\beta \in (0, 1]$ controls the decay rate over distance. The effects and selection of α and β are discussed in Appendix B.

Proxy for GU Difficulty. Next, we define a proxy metric to characterize unlearning difficulty as a unified basis for analysis. The goal is to quantify the difficulty of achieving the *trade-off of unlearning*: forgetting deleted elements ($\Delta\mathcal{G}$), preserving performance on the remaining graph ($\mathcal{G} \setminus \Delta\mathcal{G}$), and maintaining generalization on the test set $\mathcal{D}_{\text{test}}$. Existing GU studies typically evaluate these objectives separately [21, 39], lacking a unified metric to capture their joint effect. To bridge this gap, inspired by unified evaluation frameworks in related domains [30], we propose the *trade-off of unlearning* (ToU) metric, which measures the relative performance difference between the unlearned model f_{θ^u} and the retrained model f_{θ^r} across deleted elements, remaining graph, and the test set:

$$\text{ToU}(\theta^u, \theta^r, \Delta\mathcal{G}, \mathcal{G} \setminus \Delta\mathcal{G}, \mathcal{D}_{\text{test}}) = (1 - \text{diff}(\theta^u, \theta^r, \Delta\mathcal{G})) \cdot (1 - \text{diff}(\theta^u, \theta^r, \mathcal{G} \setminus \Delta\mathcal{G})) \cdot (1 - \text{diff}(\theta^u, \theta^r, \mathcal{D}_{\text{test}})), \quad (5)$$

where $\text{diff}(\theta^u, \theta^r, \mathcal{D}) = |\text{acc}(\theta^u, \mathcal{D}) - \text{acc}(\theta^r, \mathcal{D})|$ is the absolute accuracy difference between f_{θ^u} and f_{θ^r} on dataset $\mathcal{D}$. In node unlearning, for $\Delta\mathcal{G}$ and $\mathcal{G} \setminus \Delta\mathcal{G}$, $\mathcal{D}$ corresponds to their respective training samples. Intuitively, ToU favors unlearned models that behave similarly to retrained ones across all three sets. The score lies in $[0, 1]$, with higher values indicating more effective unlearning.

Relating Memorization to Unlearning Difficulty. We now investigate how GNN memorization correlates with GU difficulty. Our hypothesis is that weakly memorized samples are easier to unlearn, while highly memorized ones are harder. To test this hypothesis against the prevailing assumption that nodes closer to $\mathcal{D}_{\text{test}}$ are more difficult to unlearn, we construct five unlearning sets $\Delta\mathcal{G}$: (1) *low-mem*: the bottom $N\%$ of training nodes with the lowest $\text{mem}(\cdot)$ scores; (2) *high-mem*: the top $N\%$ with the highest scores; (3) *random*: $N\%$ of training nodes randomly sampled, following common

practice in existing GU studies; (4) *local*: the $N\%$ of training nodes closest to $\mathcal{D}_{\text{test}}$; and (5) *distant*: the $N\%$ farthest from $\mathcal{D}_{\text{test}}$.

Figure 3 presents three key observations. First, all methods perform best on the *low-mem* set and worst on the *high-mem* set, confirming them as the easiest and hardest to unlearn, respectively. Second, performance on the *random* set is closer to the *low-mem* set than to the *high-mem* set, reflecting the long-tailed nature of memorization difficulty. Third, the *distant* set is harder to unlearn than the *local* set, but both differ significantly from the *high-mem* set, indicating that distance to $\mathcal{D}_{\text{test}}$ does not fundamentally determine unlearning difficulty. These trends are consistently observed across diverse datasets and GNN backbones (see Appendix A).

Based on these results, we conclude that the memorization score is a reliable indicator of unlearning difficulty. Therefore, in node unlearning tasks, we define the difficulty of each node v_i as its memorization score, formally, $\mathcal{H}_{\text{node}}(v_i) = \text{mem}(v_i)$.

Quantifying Edge and Feature Unlearning Difficulty. We extend the memorization score to edge and feature unlearning. In node classification tasks, edges and features are not directly associated with prediction labels, making the label-based memorization score (Eq. (5)) inapplicable. To overcome this, we infer their unlearning difficulty from the memorization levels of associated nodes. The key insight is that removing edges or features alters the input information that supports label prediction. Thus, when highly memorized nodes heavily rely on the removed elements, preserving predictive performance after unlearning becomes substantially more difficult.

Formally, for each edge (v_i, v_j) , we define its unlearning difficulty by aggregating the memorization levels of its incident nodes (i.e., the two endpoints), normalized by their structural importance:

$$\mathcal{H}_{\text{edge}}(v_i, v_j) = \frac{\mathcal{H}_{\text{node}}(v_i)}{\sqrt{\deg(v_i)}} + \frac{\mathcal{H}_{\text{node}}(v_j)}{\sqrt{\deg(v_j)}}, \quad (6)$$

where $\deg(v_i)$ denotes the degree of v_i . Intuitively, edges connecting highly memorized and low-degree nodes are harder to unlearn, as their removal more severely impacts the model's predictions on the corresponding nodes. Similarly, for a feature $\mathbf{x}$, let $\mathcal{V}_{\mathbf{x}}$ be the set of training nodes associated with $\mathbf{x}$. Its unlearning difficulty is defined as the average memorization score of these nodes:

$$\mathcal{H}_{\text{feat}}(\mathbf{x}) = \frac{1}{|\mathcal{V}_{\mathbf{x}}|} \sum_{v_i \in \mathcal{V}_{\mathbf{x}}} \mathcal{H}_{\text{node}}(v_i). \quad (7)$$

We validate these metrics through experiments and demonstrate their accuracy in assessing unlearning difficulty (see Appendix A).

4.3 Adaptive Unlearning Strategy

In this section, we analyze the characteristics of unlearning targets with different difficulty levels and propose an adaptive strategy. As shown in Table 1, hard-to-unlearn nodes exhibit lower *centrality* values [3] than easy ones, indicating that they are typically located near the graph boundary. Due to sparse connectivity and weak neighbor aggregation, their features become less representative and harder to generalize. This also explains why nodes farther from the test set (i.e., *distant* nodes) tend to be more difficult to unlearn, as they are usually located in less central regions of the graph.

Although hard-to-unlearn samples lie near the graph boundary, they are crucial for GNN generalization. As shown in Figure 4, we retrain models after removing nodes of different difficulty levels and

Table 1: Centrality values of easy vs. hard-to-unlearn nodes.

Centrality Metric	Cora-GAT (N=3)			Citeseer GCN (N=5)		
	Easy	Hard	Easy/Hard	Easy	Hard	Easy/Hard
Degree	1.77e-3	9.76e-4	1.81	2.36e-3	3.50e-4	6.76
Betweenness	2.15e-3	8.55e-4	2.52	5.05e-3	3.90e-6	1295.89
Eigenvector	6.61e-3	1.38e-3	4.79	0.02	2.42e-5	777.62
PageRank	4.42e-4	3.09e-4	1.43	6.01e-4	3.16e-4	1.9
K-core	2.27	1.8	1.26	2.58	1.01	2.55

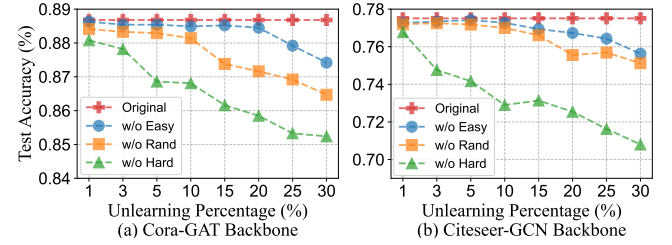


Figure 4: Impact on generalization after removing nodes at varying difficulty levels.

compare their test performance with the original model. Removing hard-to-unlearn nodes leads to substantial degradation, whereas removing the same number of random or easy-to-unlearn nodes has little effect at small unlearning ratios. This shows that unlearning targets at different difficulty levels cause varying generalization loss, challenging the common assumption that the unlearned model should preserve the same generalization as the original one [12].

A natural solution is to define different unlearning objectives based on task difficulty. However, computing difficulty scores is costly and requires recomputation after every graph modification. This raises a key question: can we design an efficient approach that dynamically adjusts unlearning objectives based on task difficulty? To this end, we propose an adaptive strategy with two components: (1) margin-based forgetting that adaptively regulates model generalization, and (2) distillation-based preservation that adaptively maintains predictive performance on the remaining graph.

Margin-based Forgetting. Prior work has shown that the *margin* between data points and decision boundaries is critical for generalization [23, 28]. In neural classifiers, the margin is typically defined as the difference between the true class score and other-class scores, where a larger margin implies stronger generalization. Building on this insight, we use margin shifts to capture generalization changes after unlearning. The key idea is that unlearning alters the model's generalization on deleted elements, with hard-to-unlearn tasks causing greater shifts than random or easy ones.

In this work, we propose a prototype-based margin adjustment strategy, where the margin is defined as the difference between a sample's distance to its true-class prototype and its distance to other-class prototypes. Traditional approaches compute prototypes by averaging embeddings, but in practical unlearning scenarios, the model may be a black box with only probability outputs available. Therefore, we represent each class prototype as the mean probability distribution of its training samples and measure distances using KL divergence. Specifically, let $\mathbf{h}_i$ denote the probability output of

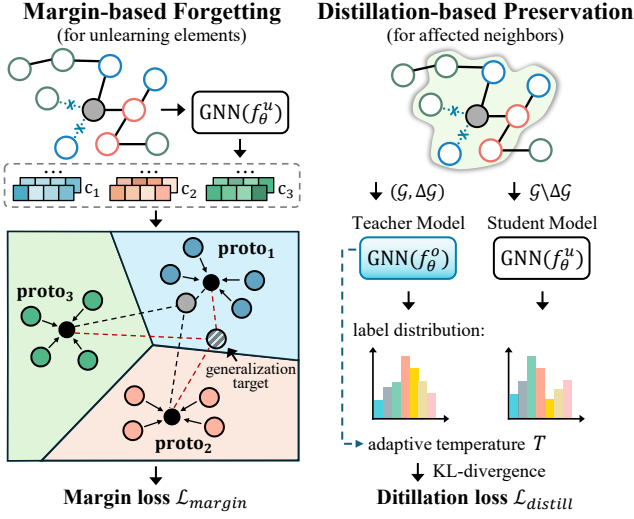


Figure 5: Adaptive Unlearning Strategy.

node v_i . For each class c , the prototype is computed as follows:

$$\text{proto}_c = \frac{1}{|S_c|} \sum_{v_i \in S_c} \mathbf{h}_i, \quad (8)$$

where S_c is the set of training nodes in class c . Then, the margin of v_i is defined as the difference between its distance to the true-class prototype and the average distance to all other-class prototypes:

$$\gamma_i = \frac{1}{|C| - 1} \sum_{c \neq y_i} \text{KL}(\mathbf{h}_i \parallel \text{proto}_c) - \text{KL}(\mathbf{h}_i \parallel \text{proto}_{y_i}), \quad (9)$$

where C is the set of all classes. Since larger margins imply stronger generalization guarantees, γ_i serves as a direct indicator of the model's generalization strength. Therefore, we design a margin-based loss as the unlearning objective for the deleted elements:

$$\begin{aligned} \mathcal{L}_{\text{margin}} &= \frac{1}{|\Delta\mathcal{G}|} \sum_{v_i \in \Delta\mathcal{G}} \max\left(0, \gamma_i^{\text{unlearn}} - (\gamma_i^{\text{ori}} - \delta_i)\right) \\ &\approx \frac{1}{|\Delta\mathcal{G}|} \sum_{v_i \in \Delta\mathcal{G}} \text{softplus}(\gamma_i^{\text{unlearn}} - \tau_i), \end{aligned} \quad (10)$$

where γ_i^{ori} and $\gamma_i^{\text{unlearn}}$ are the generalization margins before and after unlearning, δ_i is the expected margin reduction, and τ_i is the target margin after unlearning. Since GNN architectures vary widely and trained models are often black-box in practical unlearning scenarios [24], computing a universal τ_i is intractable. Thus, we treat τ_i as a learnable parameter initialized from the original model f_{θ^o} and updated during unlearning. Inspired by prior work [10, 22], which shows that the trained model f_{θ^o} provides self-supervised signals during unlearning, we freeze f_{θ^o} , feed the remaining graph $\mathcal{G} \setminus \Delta\mathcal{G}$ as input, and apply Eq. (9) to compute the initial value for each node. We replace non-differentiable $\max(\cdot)$ with its smooth approximation $\text{softplus}(\cdot)$ [47] to ensure stable optimization.

Distillation-based Preservation. Existing GU methods require the unlearned model f_{θ^u} to preserve the predictive performance of the original model f_{θ^o} on the remaining graph $\mathcal{G} \setminus \Delta\mathcal{G}$. However, they overlook that deleted elements can indirectly influence predictions on $\mathcal{G} \setminus \Delta\mathcal{G}$ by altering the model's generalization. We propose an adaptive distillation strategy that flexibly controls the extent of performance preservation. Here, the original model f_{θ^o} acts as the

teacher and the unlearned model f_{θ^u} as the student. Each node is assigned a temperature T reflecting its sensitivity to deletion: highly affected nodes use larger T to smooth the teacher's distribution and transfer richer inter-class knowledge, while less affected nodes use T close to 1 for stronger supervision. To estimate node sensitivity, we compare the outputs of f_{θ^o} on the complete graph $\mathcal{G}$ and the remaining graph $\mathcal{G} \setminus \Delta\mathcal{G}$, and compute T accordingly:

$$T_i = 1 + (T_{\max} - 1) \cdot \text{sigmoid}\left(\text{KL}(\hat{\mathbf{h}}_i \parallel \mathbf{h}_i)\right), \quad (11)$$

where $\hat{\mathbf{h}}_i$ and $\mathbf{h}_i$ denote the probability distributions of v_i under f_{θ^o} with inputs $\mathcal{G} \setminus \Delta\mathcal{G}$ and $\mathcal{G}$, respectively. $T_{\max}$ sets the upper bound of the temperature, controlling the softness of the distribution while avoiding over-smoothing. The distillation loss is then defined as:

$$\mathcal{L}_{\text{distill}} = \frac{1}{|\mathcal{G} \setminus \Delta\mathcal{G}|} \sum_{v_i \in \mathcal{G} \setminus \Delta\mathcal{G}} \text{KL}(\sigma(\mathbf{z}_i^u / T_i) \parallel \sigma(\mathbf{z}_i^o / T_i)) \cdot T_i^2, \quad (12)$$

where $\mathbf{z}_i^u$ and $\mathbf{z}_i^o$ represent the logits of the student (unlearn model) and teacher (original model), respectively, and $\sigma(\cdot)$ denotes the softmax function. The overall training objective is obtained by jointly optimizing the margin and distillation losses:

$$\mathcal{L} = \lambda \cdot \mathcal{L}_{\text{margin}} + \mathcal{L}_{\text{distill}}, \quad (13)$$

where λ is a balancing coefficient between the two objectives.

4.4 Comprehensive Evaluation Framework

In this section, we propose a difficulty-aware sampling strategy and a task-aware evaluation protocol to comprehensively evaluate the robustness, practicality, and generalizability of GU methods.

Difficulty-aware Sampling Strategy. While our difficulty assessment addresses the limitation of existing evaluations that focus on easy-to-unlearn targets and overlook hard ones, computing these scores still takes time and may not suit real-world scenarios. This motivates an efficient strategy to estimate difficulty scores.

Since our assessment relies on estimating memorization scores $\text{mem}(\cdot)$, a natural approach is to develop fast approximations and extend them to difficulty estimation. Some studies [46] approximate memorization using proxy indicators such as confidence or accuracy changes during training, but these methods require retraining and are unsuitable for graphs. A recent graph-based approach [18] estimates memorization via node homophily and heterophily, but it requires complete ground-truth labels, which are often unavailable.

To address these limitations, we propose a training-free method that estimates unlearning difficulty directly from the original model f_{θ^o} and the graph $\mathcal{G}$. Based on our finding that generalization capacity correlates with unlearning difficulty, we adopt the node margin (Eq. (9)) as a difficulty indicator, i.e., $\mathcal{H}_{\text{node}}(v_i) \approx \gamma_i$, and extend it to edge and feature difficulties via Eqs. (6) and (7). We validate this approximation through correlation analysis (see Appendix C).

Task-aware Evaluation Protocol. In Section 4.2, we propose the ToU metric (Eq. (5)) to jointly evaluate (1) forgetting ability, (2) predictive preservation, and (3) generalization capacity in node unlearning. We now extend ToU to edge and feature unlearning tasks. In node classification, these tasks do not remove node labels from $\mathcal{D}_{\text{train}}$. Instead, the goal of the unlearned model f_{θ^u} is to forget predictive patterns that previously depended on deleted edges $\Delta\mathcal{E}$ or features $\Delta\mathcal{X}$. Therefore, applying the same evaluation criteria as

Table 2: Node unlearning performance comparison. The best results are in bold, and the second-best results are underlined.

Method	Cora		Citeseer		PubMed		Photo		Computers		CS		Physics		Arxiv		Chameleon		Squirrel	
Bone	GCN	GAT	GCN	GAT	GCN	GAT	GCN	GAT	GCN	GAT	GCN	GAT	GCN	GAT	GCN	GAT	SAGE	FAGCN	SAGE	FAGCN
Easy to Unlearn																				
AGU	98.1±9	95.7±6	96.6±9	95.6±3	97.7±1	98.6±5	98.8±1	98.3±7	92.9±2	97.6±6	99.6±2	99.0±4	99.6±4	99.1±4	98.4±2	98.7±1	81.5±0	51.7±3	78.0±1	49.5±2
D2DGN	97.0±1	96.7±4	96.6±8	95.4±3	99.1±2	97.8±5	98.5±2	96.5±3	91.7±2	96.5±5	99.6±1	99.1±3	99.1±1	98.9±1	97.5±2	98.6±2	79.1±1	51.5±4	72.6±2	39.7±1
Delete	97.7±8	96.4±6	96.5±1	94.5±1	98.6±1	98.6±8	96.9±1	96.4±2	92.3±2	96.0±4	99.5±1	99.0±3	98.7±3	99.1±4	92.8±1	97.9±3	74.1±1	47.0±4	72.5±7	45.1±1
ETR	98.7±8	97.2±5	96.7±1	95.2±5	98.5±2	98.2±3	98.9±1	97.4±3	92.9±2	97.8±7	99.7±1	99.4±3	99.5±2	99.1±2	98.1±5	98.4±3	79.1±4	54.1±5	76.8±2	39.9±3
GIF	98.8±7	96.5±1	96.0±2	94.9±1	99.2±2	98.7±8	98.7±1	97.8±5	92.0±2	97.6±6	99.5±2	99.1±4	99.5±3	99.2±4	97.7±1	96.4±3	79.3±5	45.6±3	77.5±1	43.4±9
IDEA	88.4±2	96.4±2	89.2±4	93.8±7	89.4±3	96.6±7	97.3±3	98.3±5	92.9±2	96.6±4	99.6±1	99.0±3	99.2±2	98.9±2	88.8±2	91.4±3	76.5±7	48.7±4	77.6±1	44.8±9
MEGU	97.7±8	96.5±1	96.5±5	94.4±9	98.7±1	98.6±5	98.8±1	98.1±6	93.0±3	97.5±5	99.5±1	98.8±3	99.5±4	98.9±1	98.5±2	98.2±2	80.8±5	47.9±4	73.2±2	51.2±3
SGU	97.4±8	96.2±1	96.4±9	93.6±6	99.4±1	98.3±3	98.8±1	98.1±6	92.9±2	98.3±2	99.5±1	98.9±2	99.4±0	99.3±1	93.0±5	91.5±4	78.8±4	51.5±7	73.4±8	43.3±2
UTU	97.9±1	97.3±4	94.7±4	94.9±1	97.6±2	98.6±6	96.4±1	96.2±5	92.9±2	96.5±4	99.5±1	99.0±2	99.5±4	99.2±4	96.4±2	97.3±2	77.9±9	50.8±5	72.7±4	41.6±4
MGU	98.9±7	98.4±8	98.5±7	96.7±1	99.6±1	99.5±2	99.5±1	98.8±5	93.5±7	98.8±4	99.9±1	99.6±2	99.9±1	99.6±2	98.7±5	99.3±3	84.4±1	61.0±6	81.4±5	69.4±3
Improv. ¹	0.1%	1.1%	1.9%	1.2%	0.2%	0.8%	0.6%	0.5%	0.5%	0.5%	0.2%	0.2%	0.3%	0.5%	0.2%	0.6%	3.5%	12.8%	4.4%	35.6%
Random																				
AGU	88.6±9	85.2±2	80.5±4	86.0±6	97.5±8	97.3±1	97.5±2	96.0±8	95.2±6	87.1±8	99.4±3	98.4±1	98.7±2	98.8±3	97.9±3	98.1±2	42.6±1	47.5±2	50.6±3	35.4±4
D2DGN	90.8±7	82.6±1	85.4±6	86.4±4	96.7±4	96.2±7	95.5±4	97.3±9	95.6±4	88.3±5	99.3±2	97.9±2	97.9±1	97.9±1	96.1±3	96.3±2	43.8±5	49.5±2	48.3±2	34.9±3
Delete	88.6±9	85.1±6	82.1±9	91.1±3	96.4±7	97.9±6	96.5±3	96.9±2	95.1±9	89.6±1	97.8±2	98.7±4	97.0±4	98.5±4	87.7±2	97.2±1	38.4±1	47.0±3	45.0±2	34.5±2
ETR	92.7±4	86.6±1	83.9±1	93.6±2	94.5±4	98.6±4	97.3±4	97.4±1	96.2±3	88.1±1	98.7±2	98.8±5	98.4±3	98.9±6	97.7±5	97.4±6	36.9±2	57.7±1	44.2±1	36.4±3
GIF	94.3±6	86.9±2	86.5±1	90.4±3	96.3±2	96.5±8	97.3±3	96.1±1	96.0±4	87.5±6	98.4±3	98.6±6	97.2±2	98.7±3	96.2±4	94.7±8	36.4±9	51.3±2	49.2±2	36.0±3
IDEA	88.7±5	87.7±7	81.0±9	85.1±2	88.3±4	96.9±1	97.0±3	95.6±9	95.7±6	87.5±7	98.4±2	98.7±5	97.3±3	98.1±4	84.2±5	82.7±9	42.6±5	49.8±5	47.3±1	33.0±6
MEGU	91.0±7	89.3±6	86.8±9	87.1±3	97.2±5	97.0±9	95.3±4	96.3±9	96.6±5	87.5±7	99.3±2	97.9±7	98.4±4	98.2±6	98.1±3	97.0±4	39.6±8	51.7±3	42.8±1	31.6±2
SGU	94.7±1	89.1±5	84.0±7	91.8±3	97.2±4	96.4±5	97.4±2	96.0±5	96.2±2	88.0±1	99.1±1	98.4±7	98.5±1	98.5±4	88.1±6	84.1±7	40.0±2	50.5±2	40.2±1	35.5±1
UTU	92.7±9	83.8±1	81.0±6	87.8±8	96.1±4	97.1±7	96.6±4	95.8±9	95.9±4	86.1±9	98.6±2	98.2±6	98.2±2	98.6±9	94.8±4	96.0±3	37.5±1	51.6±2	45.3±2	34.0±2
MGU	97.8±3	96.0±4	95.7±1	95.7±1	98.5±2	98.8±4	98.7±4	98.6±7	97.0±1	98.5±3	99.5±1	99.2±2	99.2±1	99.6±2	98.9±5	99.1±2	60.2±6	61.7±3	69.2±8	63.7±1
Improv.	3.3%	7.5%	10.3%	2.2%	1.0%	0.2%	1.2%	1.2%	0.4%	9.9%	0.1%	0.4%	0.5%	0.7%	0.82%	1.02%	37.4%	6.9%	36.8%	75.1%
Hard to Unlearn																				
AGU	43.4±3	50.5±4	43.9±5	52.8±2	80.2±2	84.3±2	63.4±2	80.4±1	63.6±8	58.1±1	81.4±1	86.4±2	91.0±2	89.9±3	89.3±1	89.5±3	25.8±1	36.0±2	20.1±3	26.4±1
D2DGN	43.0±2	48.1±5	44.7±8	52.0±2	76.1±2	84.8±3	63.1±3	81.4±2	62.7±6	61.3±1	82.0±8	88.1±1	88.2±5	90.2±5	88.9±6	89.5±2	33.2±1	41.4±4	19.7±1	30.3±3
Delete	43.1±4	49.9±3	43.7±5	52.8±2	78.6±6	86.1±6	64.3±9	88.5±3	62.3±4	60.9±3	81.4±7	87.9±8	89.8±2	91.5±1	80.8±2	91.6±1	23.1±2	38.7±2	20.0±9	27.3±2
ETR	44.8±1	49.0±4	43.2±1	51.3±1	77.5±3	84.7±3	63.9±5	84.1±3	63.1±1	59.7±2	82.4±1	87.5±9	90.8±2	90.6±2	88.4±8	89.3±5	24.8±7	38.3±5	18.5±3	27.6±1
GIF	47.7±2	47.8±2	46.6±1	54.0±5	88.4±2	85.1±9	63.4±5	81.3±2	62.9±8	56.7±1	81.7±7	85.7±2	91.6±4	90.1±2	88.8±3	83.3±1	25.9±7	38.7±3	18.6±1	26.5±4
IDEA	65.8±5	47.4±4	61.6±3	52.5±2	87.1±4	83.9±2	63.4±4	82.4±2	63.2±7	57.4±9	81.7±9	87.6±1	90.8±6	90.1±3	90.7±2	79.7±1	24.9±2	36.6±4	18.3±3	29.2±7
MEGU	43.0±3	49.4±5	43.3±1	49.5±1	78.7±2	84.8±3	63.4±4	81.3±2	63.3±6	58.1±2	81.6±1	85.7±3	90.3±2	91.6±3	89.6±1	90.3±2	24.6±2	34.7±2	17.8±7	25.1±1
SGU	40.8±2	46.0±5	43.3±4	49.6±2	73.8±9	86.6±2	63.4±4	80.8±2	62.5±9	58.3±2	80.8±6	85.4±2	91.1±3	94.0±1	83.0±1	85.6±1	25.6±1	42.2±2	17.5±4	32.4±2
UTU	43.2±2	49.7±5	43.0±4	51.1±9	78.5±3	85.6±2	63.2±2	79.8±1	63.2±5	57.4±9	81.8±1	86.3±2	90.4±2	89.7±2	87.7±1	90.5±1	25.1±7	36.2±2	17.8±5	26.6±2
MGU	97.2±1	95.2±2	94.8±6	93.8±2	97.4±9	97.7±1	90.6±9	96.8±1	96.2±1	95.8±2	96.2±5	98.9±9	99.6±2	98.5±1	97.1±3	97.5±1	65.5±2	69.4±1	70.1±3	63.8±5
Improv.	47.7%	88.5%	53.9%	73.7%	10.2%	12.8%	40.9%	9.4%	51.3%	56.3%	16.7%	12.3%	8.7%	4.8%	7.1%	6.4%	152%	64.4%	249%	96.9%

¹ Improvements of MGU over the best-performing baseline.

in node unlearning is inappropriate, and task-specific protocols are required. Specifically, the trade-off of edge unlearning is defined as:

$$\text{ToU}_{\text{edge}}(f_{\theta^u}) = (1 - \text{diff}_{\text{MIA}}(\theta^u, \theta^r, \Delta\mathcal{E})) \cdot (1 - \text{diff}(\theta^u, \theta^r, \mathcal{D}_{\text{train}})) \cdot (1 - \text{diff}(\theta^u, \theta^r, \mathcal{D}_{\text{test}})), \quad (14)$$

where the forgetting ability $\text{diff}_{\text{MIA}}(\theta^u, \theta^r, \Delta\mathcal{E})$ is evaluated using membership inference attacks [16], which compare the capability of f_{θ^u} and f_{θ^r} to distinguish edges in $\Delta\mathcal{E}$. Predictive preservation is assessed by comparing the predictions of f_{θ^u} and f_{θ^r} on all training samples in $\mathcal{D}_{\text{train}}$, differing from existing methods [22] that assume the inference capability of nodes linked to $\Delta\mathcal{E}$ should be removed. Generalization capacity is measured by model performance on $\mathcal{D}_{\text{test}}$. Similarly, the trade-off of feature unlearning is defined as:

$$\text{ToU}_{\text{feat}}(f_{\theta^u}) = (1 - \text{diff}(\theta^u, \theta^r, \Delta\mathcal{X})) \cdot (1 - \text{diff}(\theta^u, \theta^r, \mathcal{D}_{\text{train}})) \cdot (1 - \text{diff}(\theta^u, \theta^r, \mathcal{D}_{\text{test}})), \quad (15)$$

where the forgetting ability is measured by comparing the predictions of f_{θ^u} and f_{θ^r} on $\Delta\mathcal{X}$ with and without graph structure:

$$\text{diff}(\theta^u, \theta^r, \Delta\mathcal{X}) = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} |\text{acc}(f_{\theta^u}(i)) - \text{acc}(f_{\theta^r}(i))|, \quad (16)$$

where $\mathcal{I} = \{(\mathcal{G}, \Delta\mathcal{X}), (\emptyset, \Delta\mathcal{X})\}$ denotes the two input settings (with and without graph structure). Predictive preservation is measured by comparing predictions on $\mathcal{D}_{\text{train}}$ using the remaining graph $\mathcal{G} \setminus \Delta\mathcal{X}$, while generalization capacity is evaluated on $\mathcal{D}_{\text{test}}$.

5 Experiments

We conduct extensive experiments to answer the following research questions: **RQ1**: How does MGU perform compared to state-of-the-art methods across unlearning tasks with different difficulty levels? **RQ2**: How does each proposed module in MGU contribute to the overall performance? **RQ3**: Can our adaptive unlearning strategy enhance existing methods when handling hard-to-unlearn tasks? **RQ4**: How does MGU perform under different unlearning scales and parameter settings?

5.1 Experimental Setup

Datasets. We evaluate MGU on ten widely used benchmark datasets: (1) eight homophily graphs: Cora, Citeseer, PubMed [38], Photo, Computers, CS, Physics [26], and ogbn-arxiv [17]; (2) two heterophily graphs: Chameleon and Squirrel [42]. Following recent GU studies [6, 22], we adopt a standard data split with 80% of nodes for training and 20% for testing. Detailed statistics and descriptions of these datasets are provided in Appendix D.

Backbone GNNs and Baselines. We evaluate the adaptability of MGU on five commonly used backbone GNNs: GCN [19], GAT [31], SAGE [15], GIN [37], and FAGCN [2], where FAGCN is specifically applied to heterophily graphs. We compare MGU with nine state-of-the-art baselines: AGU [10], D2DGN [27], Delete [6], ETR [39], GIF [36], IDEA [11], MEGU [22], SGU [21], and UTU [29].

Table 3: Edge and feature unlearning results on hard tasks.

Method	Citeseer		Photo		Computers		Chameleon	
Bone	SAGE	GIN	SAGE	GIN	SAGE	GIN	SAGE	GIN
Edge Unlearning								
AGU	27.8±1	77.5±9	81.5±9	46.3±2	82.0±6	69.3±0	29.6±2	62.3±9
D2DGN	27.9±1	82.1±3	81.2±3	53.6±3	84.4±1	76.7±6	33.8±3	69.8±1
Delete	28.8±8	77.6±5	81.1±7	36.3±5	81.3±4	26.8±3	27.8±2	60.4±2
ETR	27.7±1	75.6±9	81.7±6	16.5±4	82.4±9	47.9±3	30.2±9	59.4±7
GIF	34.6±3	78.0±4	81.4±8	38.6±1	81.7±7	61.9±6	29.3±2	57.1±3
IDEA	35.6±5	81.8±1	81.2±6	29.4±3	81.6±5	47.9±6	30.1±1	55.3±4
MEGU	27.7±2	76.6±7	82.0±1	37.7±7	81.1±4	63.0±4	29.2±8	61.7±4
SGU	42.2±7	78.4±8	80.7±2	46.2±5	81.2±4	78.6±9	30.0±9	68.4±1
UTU	27.8±2	77.5±2	81.8±8	33.4±2	81.9±1	52.7±6	29.8±4	62.8±2
MGU	94.5±4	90.8±1	93.2±6	83.6±2	93.9±2	80.8±2	45.8±3	71.9±6
Improv.	124%	10.6%	13.7%	56.0%	11.3%	2.8%	35.5%	3.0%
Feature Unlearning								
AGU	33.6±4	77.7±3	62.0±2	4.3±6	60.0±1	47.1±5	12.1±6	59.4±1
D2DGN	23.4±2	76.6±4	60.9±1	63.1±9	59.9±6	54.3±3	28.4±2	80.1±7
Delete	2.9±0	84.8±5	61.5±2	7.3±2	59.6±5	14.6±3	11.8±8	72.5±6
ETR	18.2±6	82.7±7	60.8±2	25.3±2	60.0±9	42.9±5	11.8±1	57.6±4
GIF	14.1±7	76.1±8	61.9±2	65.5±2	59.1±1	61.6±7	11.8±4	59.0±4
IDEA	2.9±0	74.8±2	63.8±4	68.4±5	60.2±6	52.2±3	11.7±2	51.1±3
MEGU	2.9±0	78.4±3	62.8±4	70.9±3	59.6±7	64.8±2	11.5±3	64.3±8
SGU	18.7±4	75.6±1	59.0±6	53.1±7	57.6±7	62.6±4	11.5±4	73.7±7
UTU	23.7±5	81.5±4	62.3±7	60.8±4	59.7±8	63.4±2	11.7±3	77.8±4
MGU	87.4±3	91.1±1	80.8±7	89.0±9	74.0±5	73.1±1	70.1±2	82.2±2
Improv.	160%	7.4%	26.6%	25.5%	22.9%	12.8%	146%	2.6%

Detailed descriptions of the GNNs and baselines are provided in Appendix. For all methods, we fix the embedding dimension at 64 and use 2 GNN layers. Baseline parameters are initialized with the settings reported in the original papers and further fine-tuned for optimal performance. To ensure a fair comparison, each experiment is repeated ten times and the average results are reported.

Unlearning Tasks. We evaluate all methods under three difficulty settings (*easy*, *random*, and *hard*). The *easy* and *hard* settings select the bottom- and top-ranked elements by memorization scores, respectively, while the *random* setting uniformly samples elements following the common protocol in existing work [12]. Each setting is applied to three types of GU tasks: (1) *Node unlearning*, where 5% of nodes and their connected edges are deleted; (2) *Edge unlearning*, where 5% of edges are deleted; and (3) *Feature unlearning*, where the features of 5% of nodes are replaced with zeros. This yields nine evaluation scenarios (three difficulty settings $\times$ three tasks). Results for additional unlearning ratios are provided in Appendix H.

Evaluation Metrics. We adopt the proposed *Trade-off of Unlearning* (ToU) as the primary evaluation metric. In addition, we evaluate whether GU methods effectively prevent information leakage by applying two commonly used attack strategies: membership inference attack [44] and poisoning edge attack [36], at node and edge levels. Detailed attack configurations are provided in Appendix G.

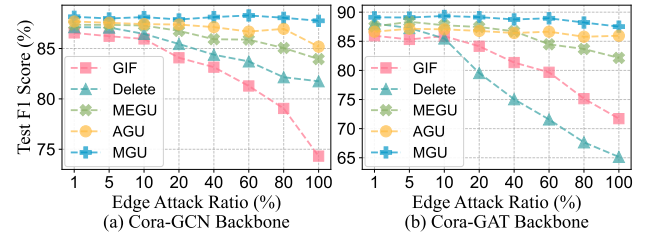
5.2 Performance Comparison (RQ1)

We compare the performance of all methods from three perspectives: *effectiveness*, *efficiency*, and *adversarial robustness*.

Effectiveness. Tables 2 and 3 report the ToU performance across GU tasks of different difficulty levels and backbone GNNs. MGU consistently outperforms all baselines across all settings. On average, it achieves improvements of 3.29% on easy tasks (0.1% to

Table 4: Average runtime of node unlearning (in seconds).

Method	Cora	Citeseer	PubMed	CS	Physics	Arxiv	Cham.	Squi.
GCN								
Retrain	0.71	0.53	0.96	2.06	2.93	16.18	0.52	0.88
ETR	0.02	0.02	0.02	0.02	0.03	0.03	0.02	0.01
GIF	0.14	0.11	0.11	0.32	0.73	2.08	0.10	0.27
IDEA	0.10	0.11	0.11	0.32	0.73	2.13	0.10	0.21
AGU	0.06	0.06	0.06	0.12	0.24	0.48	0.07	0.09
D2DGN	0.16	0.18	0.17	0.26	0.51	1.14	0.16	0.20
Delete	1.37	1.23	1.31	2.47	4.51	39.25	1.51	3.01
MEGU	0.09	0.08	0.08	0.14	0.26	0.51	0.09	0.12
SGU	0.47	0.55	2.79	2.60	5.44	28.34	0.42	0.83
MGU	0.02	0.01	0.03	0.07	0.04	0.20	0.04	0.05
GAT								
Retrain	0.77	0.55	0.82	2.88	4.02	30.04	0.63	1.84
ETR	0.03	0.02	0.02	0.03	0.05	0.04	0.02	0.02
GIF	0.35	0.32	0.35	0.62	1.39	82.27	0.31	3.46
IDEA	0.36	0.34	0.35	0.63	1.44	86.38	0.32	3.32
AGU	0.09	0.08	0.10	0.21	0.43	1.05	0.10	0.13
D2DGN	0.22	0.24	0.24	0.46	0.84	1.85	0.24	0.30
Delete	1.85	1.82	2.03	4.05	7.79	13.17	1.90	1.95
MEGU	0.11	0.11	0.13	0.23	0.45	1.03	0.13	0.17
SGU	0.50	0.58	2.89	2.74	6.00	25.43	0.46	0.84
MGU	0.02	0.02	0.04	0.11	0.23	0.15	0.02	0.03

**Figure 6: PEA performance under edge unlearning.**

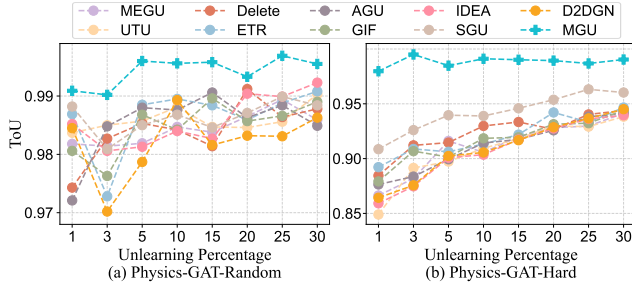
35.6%), 9.82% on random tasks (0.1% to 75.1%), and 53.1% on hard tasks (2.6% to 249%) over the best-performing baselines. These results demonstrate MGU’s robustness across diverse difficulty levels, task types, backbone GNNs, and graph types (homophily and heterophily), highlighting the advantage of its adaptive unlearning strategy. In contrast, baseline methods adopt uniform unlearning schemes, leading to inferior performance, particularly on hard tasks.

Efficiency. Table 4 reports the average unlearning time. MGU significantly outperforms learning-based baselines (AGU, D2DGN, Delete, MEGU, and SGU), achieving speedups of 3.5 $\times$ to 70 $\times$. Compared with training-free methods (GIF, IDEA, ETR), which directly adjust model parameters without training, MGU remains highly competitive, ranking second only to ETR in some cases. However, ETR requires additional preprocessing time to extract subgraphs based on unlearning requests. Moreover, our experiments show that training-free methods are sensitive to hyperparameters and may cause model collapse on the test set. In contrast, MGU maintains robust and stable performance with only 10–30 training epochs.

Adversarial Robustness. Figure 6 shows the robustness of GU methods against poisoning edge attacks (PEA). Across both GCN and GAT backbones, MGU exhibits superior resistance to adversarial perturbations, maintaining stable performance as the attack ratio increases. In contrast, GIF and Delete experience substantial

Table 5: Node unlearning performance of MGU variants.

GNN Bone	Method	PubMed			Photo		
		Easy	Random	Hard	Easy	Random	Hard
GCN	w/o Margin	96.5±.8	95.5±.9	82.9±.6	98.7±.3	97.5±.2	77.4±.4
	w/o Distill	97.1±.5	94.6±.3	83.6±.7	95.6±.1	87.3±.8	76.3±.1
	MGU	99.6±.1	98.7±.4	90.6±.9	99.5±.1	98.7±.4	90.6±.9
GAT	w/o Margin	98.1±.2	95.3±.6	88.2±.1	97.6±.6	95.5±.5	87.8±.4
	w/o Distill	97.7±.8	93.4±.5	81.2±.4	94.8±.7	90.2±.5	88.6±.7
	MGU	99.5±.2	98.8±.4	97.7±.1	98.8±.5	98.6±.7	96.8±.1
SAGE	w/o Margin	96.4±.9	92.5±.9	59.6±.8	97.7±.2	93.6±.8	72.6±.2
	w/o Distill	95.8±.1	91.9±.7	68.3±.4	97.8±.5	96.5±.5	81.4±.7
	MGU	99.5±.4	98.2±.8	76.9±.1	99.6±.1	97.8±.3	92.4±.3

**Figure 7: Node unlearning under different ratios.**

degradation, with Delete showing particularly sharp drops on GAT. Results for membership inference attacks (MIA) are provided in Appendix H, where MGU consistently outperforms all baselines.

5.3 Ablation Study (RQ2)

We conduct ablation studies to evaluate MGU’s key components by constructing two variants: (1) **w/o Margin** removes the margin-based forgetting module, and (2) **w/o Distill** excludes the distillation-based preservation module. Table 5 summarizes the results across three GNN backbones under different difficulty settings. MGU consistently outperforms both variants, with the largest gap in hard tasks: removing the margin module leads to an average 8.5% drop, while excluding distillation causes a 7.8% drop. In contrast, the performance gaps on easy and random settings are much smaller (1.2% and 3.1%, respectively). These results highlight the importance of both modules, especially for handling hard-to-unlearn cases.

5.4 Strategy Generalizability (RQ3)

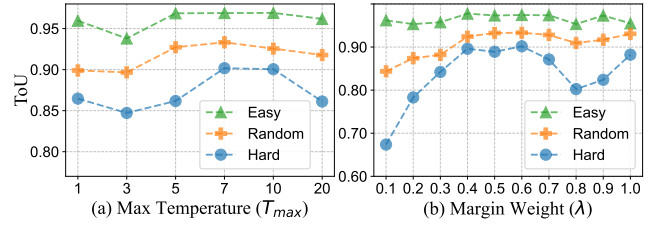
We evaluate whether our adaptive unlearning strategy (AUS) can enhance existing methods on hard-to-unlearn tasks. Table 6 reports the results of integrating our strategy (+AUS) into five baseline methods under hard settings. On average, AUS improves performance by 47.8% on homophily graphs (Cora and Computers) and 44.0% on the heterophily graph (Chameleon). These results demonstrate the effectiveness and applicability of our strategy across diverse graph structures and baseline methods.

5.5 Scalability and Parameter Analysis (RQ4)

We evaluate the performance of MGU under different unlearning scales and parameter settings. Figure 7 presents ToU results with

Table 6: Baseline performance improvements on hard tasks.

Method	Cora		Computers		Chameleon		Improv.
	GCN	GAT	GCN	GAT	SAGE	FAGCN	
AGU	43.4±.3	50.5±.4	63.6±.8	50.5±.4	25.8±.1	36.0±.2	61.6%
+AUS	81.9±.7	84.2±.6	83.4±.5	74.4±.9	52.3±.1	47.9±.3	
D2DGN	43.0±.2	48.1±.5	62.7±.6	61.3±.1	33.2±.1	41.4±.4	21.5%
+AUS	51.3±.8	57.8±.3	68.6±.1	62.8±.3	55.2±.3	46.2±.3	
MEGU	43.0±.3	49.4±.5	63.4±.4	81.3±.2	24.6±.2	34.7±.2	52.4%
+AUS	79.8±.3	86.8±.9	74.1±.7	83.2±.7	41.2±.3	57.8±.8	
SGU	44.8±.1	49.0±.4	63.1±.1	59.7±.2	24.8±.7	38.3±.5	40.4%
+AUS	56.8±.4	55.1±.3	78.2±.8	81.4±.1	52.3±.1	50.6±.3	
UTU	43.2±.2	49.7±.5	63.2±.5	57.4±.9	25.1±.7	36.2±.2	50.6%
+AUS	74.3±.2	82.3±.3	72.2±.9	62.7±.3	53.7±.1	46.5±.3	

**Figure 8: Feature unlearning performance under varying parameters on Photo with GCN.**

unlearning ratios from 1% to 30%. MGU consistently exhibits superior and stable performance across all scales, demonstrating strong robustness to diverse unlearning requests. In contrast, baseline methods perform significantly worse on hard tasks than on random ones, highlighting the difficulty of unlearning highly memorized elements. Interestingly, as the unlearning ratio increases, the performance gap between unlearned and retrained models narrows, suggesting that deleting more elements removes essential structural information that both models rely on.

Figure 8 presents the sensitivity analysis of two key parameters $T_{\max}$ and λ . $T_{\max}$ controls the maximum softening in distillation. Performance on easy and random tasks remains stable when $T_{\max}$ ranges from 5 to 10, whereas hard tasks require larger values (7–10) due to their stronger impact on generalization. The margin loss weight λ shows a similar trend: performance on easy and random settings remains relatively stable across parameter ranges, while performance on hard settings is more sensitive. Overall, setting λ between 0.4 and 0.7 provides robust and consistent performance.

6 Conclusion

In this paper, we re-understand graph unlearning from a new perspective of GNN memorization and propose MGU, a memorization-guided unlearning framework that accurately assesses unlearning difficulty and adaptively adjusts unlearning objectives. Extensive experiments demonstrate the superior performance of MGU in forgetting quality, computational efficiency, and utility preservation. Future work will explore memorization mechanisms in diverse graph types and their unlearning implications.

ACKNOWLEDGMENTS

This research is supported by ARC Discovery Project DP230100676.

References

- [1] Devansh Arpit, Stanislaw Jastrzębski, Nicolas Ballas, David Krueger, Emmanuel Bengio, et al. 2017. A Closer Look at Memorization in Deep Networks. In *ICML*. PMLR, 233–242.
- [2] Deyu Bo, Xiao Wang, Chuan Shi, and Huawei Shen. 2021. Beyond Low-Frequency Information in Graph Convolutional Networks. In *AAAI*, Vol. 35. 3950–3957.
- [3] Stephen P. Borgatti and Martin G. Everett. 2006. A Graph-Theoretic Perspective on Centrality. *Social Networks* 28, 4 (2006), 466–484.
- [4] Min Chen, Zhikun Zhang, Tianhao Wang, Michael Backes, Mathias Humbert, and Yang Zhang. 2022. Graph Unlearning. In *CCS*. ACM, 499–513.
- [5] Ziheng Chen, Jiali Cheng, Gabriele Tolomei, Sijia Liu, Hadi Amiri, Yu Wang, Kaushiki Nag, and Lu Lin. 2025. FROG: Fair Removal on Graphs. *arXiv preprint*. arXiv:2503.18197
- [6] Jiali Cheng, George Dasoulas, Huan He, and Chirag Agarwal. 2023. GNNDelete: A General Strategy for Unlearning in Graph Neural Networks. In *ICLR*.
- [7] Eli Chien, Chao Pan, and Olga Milenkovic. 2023. Efficient Model Updates for Approximate Unlearning of Graph-Structured Data. In *ICLR*.
- [8] Weilin Cong and Mehrdad Mahdavi. 2022. GraphEditor: An Efficient Graph Representation Learning and Unlearning Approach. In *NeurIPS*.
- [9] Joost C. F. De Winter, Samuel D. Gosling, and Jeff Potter. 2016. Comparing the Pearson and Spearman Correlation Coefficients across Distributions and Sample Sizes: A Tutorial Using Simulations and Empirical Data. *Psychological Methods* 21, 3 (2016), 273–290.
- [10] Pengfei Ding, Yan Wang, Guanfeng Liu, and Jiajie Zhu. 2025. Adaptive Graph Unlearning. In *IJCAI*. 2767–2774.
- [11] Yushun Dong, Binchi Zhang, Zhenyu Lei, Na Zou, and Jundong Li. 2024. IDEA: A Flexible Framework of Certified Unlearning for Graph Neural Networks. In *KDD*. ACM, 621–630.
- [12] Bowen Fan, Yuming Ai, Xunkai Li, Zhilin Guo, Rong-Hua Li, and Guoren Wang. 2025. OpenGU: A Comprehensive Benchmark for Graph Unlearning. *arXiv preprint*. arXiv:2501.02728
- [13] Vitaly Feldman and Chiyuan Zhang. 2020. What Neural Networks Memorize and Why: Discovering the Long Tail via Influence Estimation. In *Advances in Neural Information Processing Systems*, Vol. 33. 2881–2891.
- [14] Zhiwei Guo and Heng Wang. 2020. A Deep Graph Neural Network-Based Mechanism for Social Recommendations. *IEEE Transactions on Industrial Informatics* 17, 4 (2020), 2776–2783.
- [15] Will Hamilton, Zitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *NeurIPS*.
- [16] Hongsheng Hu, Zoran Salic, Lichao Sun, Gillian Dobbie, Philip S. Yu, and Xuyun Zhang. 2022. Membership Inference Attacks on Machine Learning: A Survey. *Comput. Surveys* 54, 11s (2022), 1–37.
- [17] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. In *NeurIPS*.
- [18] Adarsh Jamadandi, Jing Xu, Adam Dziedzic, and Franziska Boenisch. 2025. Memorization in Graph Neural Networks. *arXiv preprint*. arXiv:2508.19352
- [19] Thomas N. Kipf and Max Welling. 2016. Semi-Supervised Classification with Graph Convolutional Networks. *arXiv preprint*. arXiv:1609.02907
- [20] Fan Li, Xiaoyang Wang, Dawei Cheng, Wenjie Zhang, Ying Zhang, and Xuemin Lin. 2024. TCGU: Data-Centric Graph Unlearning Based on Transferable Condensation. *arXiv preprint*. arXiv:2410.06480
- [21] Xunkai Li, Bowen Fan, Zhengyu Wu, Zhiyu Li, Rong-Hua Li, and Guoren Wang. 2025. Toward Scalable Graph Unlearning: A Node Influence Maximization Based Approach. *arXiv preprint*. arXiv:2501.11823
- [22] Xunkai Li, Yulin Zhao, Zhengyu Wu, Wentao Zhang, Rong-Hua Li, and Guoren Wang. 2024. Towards Effective and General Graph Unlearning via Mutual Evolution. In *AAAI*, Vol. 38. 13682–13690.
- [23] Weiyang Liu, Yandong Wen, Zhiding Yu, and Meng Yang. 2016. Large-Margin Softmax Loss for Convolutional Neural Networks. In *ICML*. 507–516.
- [24] Subhodip Panda and A. P. Prathosh. 2024. FAST: Feature-Aware Similarity Thresholding for Weak Unlearning in Black-Box Generative Models. *IEEE Transactions on Artificial Intelligence* (2024).
- [25] Kartik Sharma, Yeon-Chang Lee, Sivagami Nambi, Aditya Salian, Shlok Shah, Sang-Wook Kim, and Srikanth Kumar. 2024. A Survey of Graph Neural Networks for Social Recommender Systems. *Comput. Surveys* 56, 10 (2024), 1–34.
- [26] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. 2018. Pitfalls of Graph Neural Network Evaluation. In *NeurIPS Workshop on Relational Representation Learning (R2L)*.
- [27] Yash Sinha, Murari Mandal, and Mohan Kankanahalli. 2023. Distill to Delete: Unlearning in Graph Networks with Knowledge Distillation. *arXiv preprint*. arXiv:2309.16173
- [28] Jaeyun Song, Joonhyung Park, and Eunho Yang. 2022. TAM: Topology-Aware Margin Loss for Class-Imbalanced Node Classification. In *ICML*. 20369–20383.
- [29] Jiajun Tan, Fei Sun, Ruichen Qiu, Du Su, and Huawei Shen. 2024. Unlink to Unlearn: Simplifying Edge Unlearning in GNNs. In *TheWebConf*. 489–492.
- [30] Eleni Triantafillou, Peter Kairouz, Fabian Pedregosa, Jamie Hayes, et al. 2024. Are We Making Progress in Unlearning? Findings from the First NeurIPS Unlearning Competition. *arXiv preprint*. arXiv:2406.09073
- [31] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2017. Graph Attention Networks. *arXiv preprint*. arXiv:1710.10903
- [32] Cheng-Long Wang, Mengdi Huai, and Di Wang. 2023. Inductive Graph Unlearning. In *USENIX Security*. 3205–3222.
- [33] Jianan Wang, Sheng Zhang, Yanguhua Xiao, and Rui Song. 2021. A Review on Graph Neural Network Methods in Financial Applications. *arXiv preprint*. arXiv:2111.15367
- [34] Kuansan Wang, Zhihong Shen, Chiyuan Huang, Chieh-Han Wu, Yuxiao Dong, and Anshul Kanakia. 2020. Microsoft Academic Graph: When Experts Are Not Enough. *Quantitative Science Studies* 1, 1 (2020), 396–413.
- [35] Jiaheng Wei, Yanjun Zhang, Leo Zhang, Ming Ding, Chao Chen, Kok-Leong Ong, Jun Zhang, and Yang Xiang. 2024. Memorization in Deep Learning: A Survey. *Comput. Surveys* (2024).
- [36] Jiancan Wu, Yi Yang, Yuchun Qian, Yongduo Sui, Xiang Wang, and Xiangnan He. 2023. GIF: A General Graph Unlearning Strategy via Influence Function. In *TheWebConf*. 651–661.
- [37] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How Powerful Are Graph Neural Networks? *arXiv preprint*. arXiv:1810.00826
- [38] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting Semi-Supervised Learning with Graph Embeddings. In *ICLR*. 40–48.
- [39] Zhe-Rui Yang, Jindong Han, Chang-Dong Wang, and Hao Liu. 2025. Erase then Rectify: A Training-Free Parameter Editing Approach for Cost-Effective Graph Unlearning. In *AAAI*.
- [40] Lu Yi and Zhewei Wei. 2024. Scalable and Certifiable Graph Unlearning via Lazy Local Propagation. *arXiv preprint*. arXiv:2408.09212
- [41] Lijia Yu, Xiao-Shan Gao, Lijun Zhang, and Yibo Miao. 2024. Generalizability of Memorization Neural Network. In *NeurIPS*.
- [42] Hanqing Zeng, Hongkuan Zhou, Ajitesh Srivastava, Rajgopal Kannan, and Viktor Prasanna. 2019. GraphSAINT: Graph Sampling Based Inductive Learning Method. In *ICLR*.
- [43] Jiahao Zhang. 2024. Graph Unlearning with Efficient Partial Retraining. In *TheWebConf*. 1218–1221.
- [44] Zhikun Zhang, Min Chen, Michael Backes, Yun Shen, and Yang Zhang. 2022. Inference Attacks Against Graph Neural Networks. In *USENIX Security*. 4543–4560.
- [45] Kairan Zhao, Meghdad Kurmanji, George-Octavian Bărbulescu, Eleni Triantafillou, and Peter Triantafillou. 2024. What Makes Unlearning Hard and What to Do About It. In *NeurIPS*.
- [46] Kairan Zhao and Peter Triantafillou. 2024. Scalability of Memorization-based Machine Unlearning. In *NeurIPS 2024 Workshop on Fine-Tuning in Modern Machine Learning: Principles and Scalability*.
- [47] Hao Zheng, Zhanlei Yang, Wenju Liu, Jizhong Liang, and Yanpeng Li. 2015. Improving Deep Neural Networks Using Softplus Units. In *IJCNN*. 1–4.

A Extended Evaluation of GU Methods on Hard-to-unlearn Tasks

To further verify the ineffectiveness of existing GU methods on hard-to-unlearn tasks and validate the rationality of our proposed edge and feature unlearning difficulty formulations (Eqs. (6) and (7)), we conduct additional experiments following the settings in Section 4.2, using more datasets and diverse GNN backbones to provide comprehensive and consistent evidence.

The results are shown in Figures 9 and 10. For both edge and feature unlearning, existing GU methods perform worst on the high-mem set and best on the low-mem set, following the same trend observed in node unlearning (Figure 3). The random set exhibits performance similar to the low-mem set, reflecting the long-tailed nature of unlearning difficulty. Although the distant set is harder to unlearn than the local set, both remain much easier than the high-mem set, indicating that spatial distance to the test set is not the dominant factor. These consistent results across datasets and GNN backbones confirm the generality of our findings and the validity of the proposed edge and feature unlearning difficulty metrics.

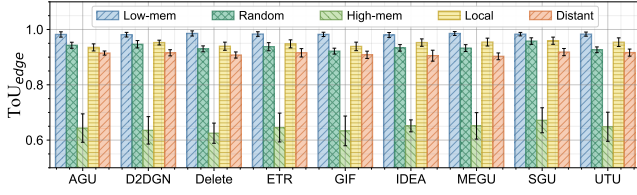


Figure 9: ToU performance of baselines for edge unlearning with SAGE backbone on PubMed dataset ($N = 3$).

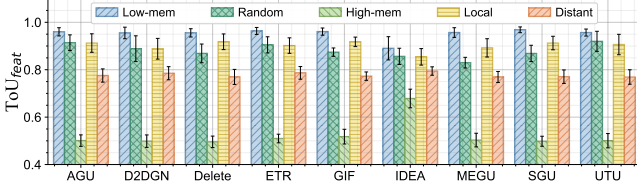


Figure 10: ToU performance of baselines for feature unlearning with GCN backbone on Citeseer dataset ($N = 10$).

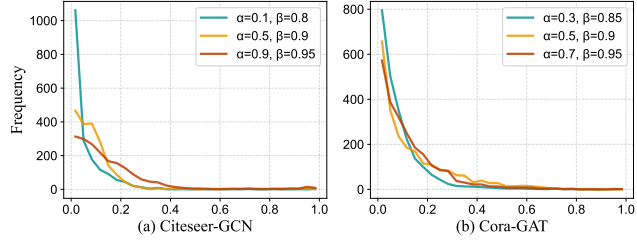


Figure 11: Difficulty distribution of node unlearning.

B Impact of Parameters on GU Difficulty

We conducted experiments to analyze the unlearning difficulty under different settings of α and β . As shown in Figure 11, varying these parameters has little impact on the long-tailed region of the distribution. However, appropriate choices of α and β enhance the separability in the low-difficulty region, allowing for a more precise distinction among these nodes. Considering their practical interpretations, which reflect the relative influence of central nodes and neighbors, and the marginal diminishing effect, we set $\alpha = 0.5$ and $\beta = 0.9$ in our experiments.

C Correlation Analysis for Validating the Margin-based Difficulty Estimation

We evaluate the effectiveness of our margin-based difficulty estimation (Section 4.4) through correlation analysis. The goal is to verify whether the estimated difficulty scores align with those obtained from memorization-based evaluations (Section 4.2). For comparison, we include several representative proxy-based memorization estimation metrics. Following [45], we adopt four indicators commonly used in computer vision, including *confidence*, *max confidence*, *entropy*, and *binary accuracy*. In addition, we incorporate the *homophily* estimator proposed by [18], which measures label consistency among neighboring nodes in graphs.

Table 7: Comparison of proxies based on Spearman correlation with unlearning difficulty.

Proxy	Cora		PubMed		Chameleon	
	GCN	GAT	GCN	GAT	GCN	GAT
Confidence	-0.6533	-0.6885	-0.7266	-0.6472	0.6492	0.5989
Max confidence	-0.6534	-0.6917	-0.7401	-0.6734	0.6935	0.5665
Entropy	-0.6415	-0.6900	-0.7350	-0.6732	0.7165	0.5546
Binary accuracy	-0.2878	-0.1608	-0.3289	-0.2314	0.2709	0.4422
Homophily	-0.4119	-0.4059	-0.4378	-0.3527	-0.0475	0.0431
Margin	-0.7886	-0.7841	-0.8033	-0.7299	-0.7412	-0.7608

Table 8: Statistics of Datasets.

Dataset	#Nodes	#Edges	#Features	#Classes
Cora	2,708	5,429	1,433	7
Citeseer	3,327	4,732	3,703	6
PubMed	19,717	44,338	500	3
Photo	7,487	119,043	745	8
Computers	13,381	245,778	767	10
CS	18,333	81,894	6,805	15
Physics	34,493	247,962	8,415	5
Arxiv	169,343	1,166,243	128	40
Chameleon	2,277	36,101	2,325	5
Squirrel	5,201	216,933	2,089	5

We compute the Spearman correlation coefficients [9] between each estimated difficulty metric and our proposed unlearning difficulty measure ($\mathcal{H}(\cdot)$). As shown in Table 7, the margin-based estimator achieves the highest correlations across all settings, clearly outperforming both training-dependent and label-dependent proxies. These results demonstrate that the margin value is a reliable and efficient proxy for assessing unlearning difficulty in a fully training-free and label-independent manner. In contrast, training-dependent metrics such as *confidence* and *entropy* exhibit opposite correlation trends on homophily (Cora) and heterophily (Chameleon) graphs, indicating their unsuitability for graph data. Moreover, the *homophily* metric shows consistently low correlations, suggesting its limited capacity to capture unlearning difficulty in graphs.

D Dataset Descriptions

We conduct experiments on ten publicly available benchmark graph datasets: Cora, Citeseer, PubMed, Photo, Computers, CS, Physics, ogbn-arxiv, Chameleon, and Squirrel. Table 8 summarizes their statistics, and detailed dataset descriptions are presented below.

Cora, **CiteSeer**, and **PubMed** [38] are benchmark citation networks, where nodes represent research papers and edges denote citation links. Node features are binary bag-of-words vectors, and labels indicate the research topics of the papers.

Photo and **Computers** [26] are two subgraphs of the Amazon co-purchase network, where nodes represent products and edges indicate frequent co-purchase relationships. Each product node is described by bag-of-words features extracted from product reviews, and the learning task is to classify products into their categories.

CS and **Physics** [26] are co-authorship networks derived from the Microsoft Academic Graph. Nodes represent authors, and edges

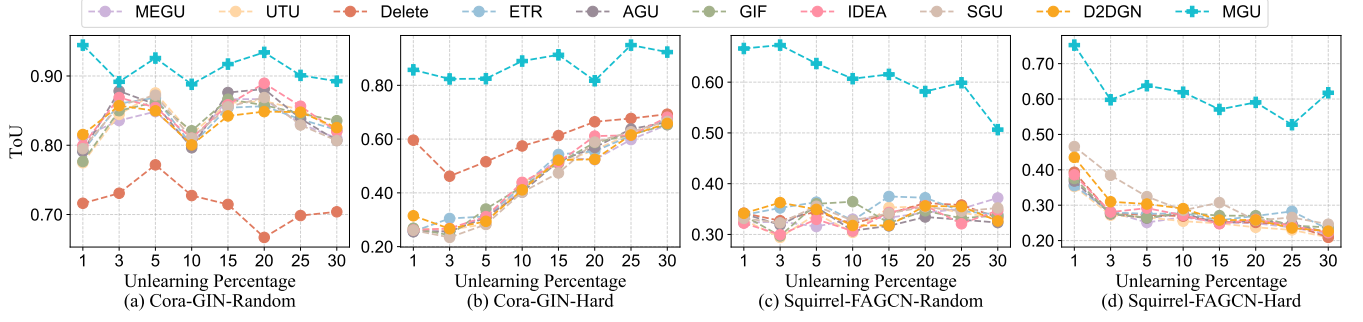


Figure 12: Node unlearning performance under different unlearning ratios.

connect authors who have co-authored at least one paper. Node features are built from paper keywords in each author’s publications, and class labels denote the author’s primary research field.

ogbn-products [17] is a large-scale citation network derived from the Microsoft Academic Graph (MAG) [34], containing a subset of computer science papers from arXiv. Nodes represent papers and edges denote citation links. It serves as a representative benchmark for evaluating graph neural networks on large academic graphs.

Chameleon and **Squirrel** [42] are Wikipedia-based webpage networks, where nodes correspond to pages and edges represent mutual hyperlinks. Node features are derived from webpage content. These datasets are widely used for evaluating heterophilic graph learning methods.

E Backbone GNNs

We select five widely used GNNs as backbone models to simulate scenarios where unlearning requests arise during training. Brief descriptions of these backbones are presented below.

- **GCN** [19] formulates message passing as a first-order approximation of spectral graph convolutions, which enables efficient and scalable learning on graph-structured data.
- **GAT** [31] leverages a self-attention mechanism to adaptively assign weights to neighboring nodes during feature aggregation.
- **SAGE** [15] adopts an inductive framework that learns node representations by aggregating attribute information from sampled neighbors, enabling generalization to unseen nodes.
- **GIN** [37] employs a multi-layer perceptron (MLP)-based aggregation scheme that achieves representational power equivalent to the Weisfeiler–Lehman graph isomorphism test.
- **FAGCN** [2] employs a frequency-adaptive convolution mechanism that dynamically balances low- and high-frequency signals, enabling effective learning on heterophilic graphs.

F Graph Unlearning Baselines

We provide brief descriptions of the state-of-the-art graph unlearning baselines used in our experiments.

- **AGU** [10] introduces an adaptive graph unlearning framework that integrates task-adaptive element forgetting across nodes, edges, and features with GNN-aware neighbor selection.
- **D2DGN** [27] proposes a model-agnostic distillation framework that separates knowledge into retention and deletion parts for efficient unlearning.

- **Delete** [6] introduces a model-agnostic, layer-wise optimization operator that adjusts the topological influence of deleted elements to achieve effective graph unlearning.
- **ETR** [39] introduces a neighborhood-aware parameter editing strategy and a subgraph-based gradient approximation to mitigate the influence of unlearned samples.
- **GIF** [36] leverages influence functions to approximate parameter updates induced by unlearning requests, enabling efficient and targeted graph unlearning.
- **IDEA** [11] analyzes objective discrepancies before and after information removal and provides theoretical guarantees for effective graph unlearning.
- **MEGU** [22] unifies predictive performance and forgetting capability within a single optimization framework, enabling mutual enhancement between the two objectives.
- **UTU** [29] mitigates the influence of forgotten edges by unlinking them, thereby blocking corresponding message-passing paths during inference.
- **SGU** [21] introduces an influence maximization-based module for scalable identification of high-impact entities and performs entity-aware fine-tuning within a unified unlearning framework.

G Membership Inference and Poisoning Attack Settings

As introduced in the main text, we employ two representative attack strategies to evaluate information leakage at both the node and edge levels. In this section, we provide detailed configurations and implementation settings for these attacks.

Membership Inference Attack (MIA). Following [44], we train a binary inference model to distinguish whether a given node was part of the training data. Specifically, we train a logistic regression classifier as the attacker. The attack is performed on both the original and unlearned models to measure the residual memorization of deleted nodes. The attack performance is reported in terms of AUC, where lower values indicate weaker membership distinguishability.

However, since each model inherently exhibits prediction bias, directly interpreting an AUC value close to 0.5 as evidence of effective unlearning can be misleading. Therefore, we further compare the AUC gap between the unlearned and the retrained model. Specifically, we define $1 - |AUC_{\text{unlearn}} - AUC_{\text{retrain}}|$ as a relative consistency metric, where higher values indicate stronger unlearning effectiveness and lower information leakage.

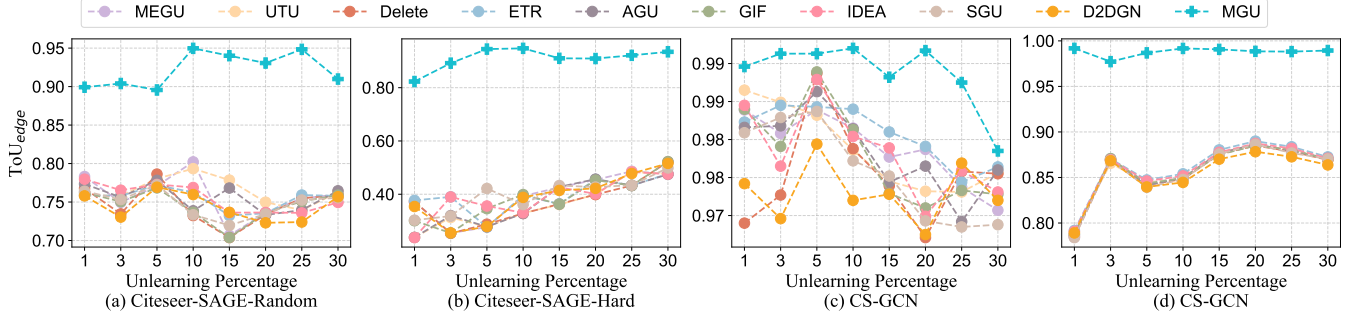


Figure 13: Edge unlearning performance under different unlearning ratios.

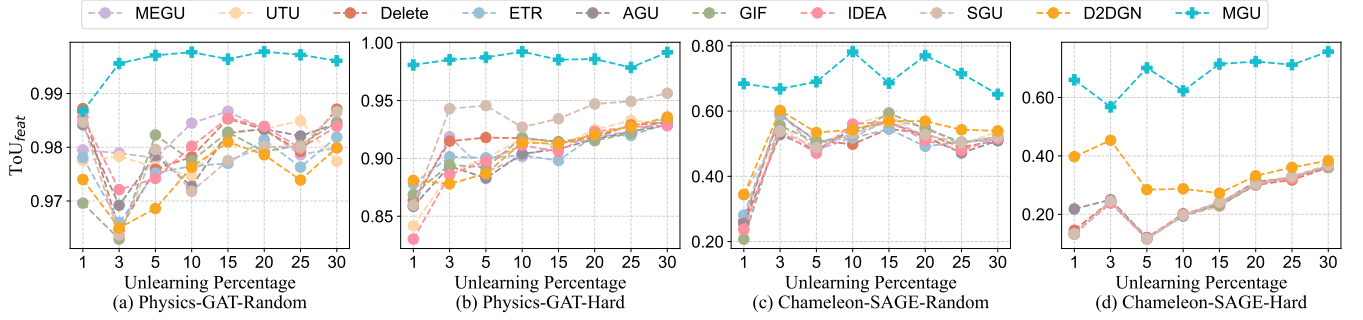


Figure 14: Feature unlearning performance under different unlearning ratios.

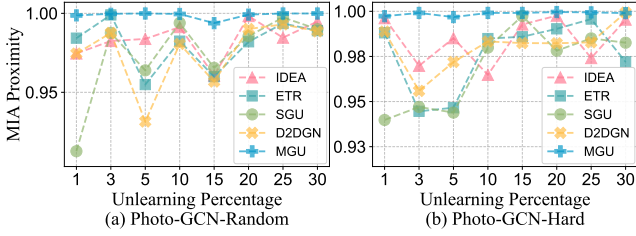


Figure 15: MIA performance under node unlearning.

Poisoning Edge Attack (PEA). Following previous works [22, 36], we simulate an edge-level poisoning attack by injecting noisy edges into the training graph to mislead representation learning. Each injected edge connects two nodes from different classes to maximize its adversarial impact. These poisoned edges are subsequently treated as unlearning elements. The unlearning capability of GU methods is then evaluated by their effectiveness in mitigating the negative impact of these poisoned edges on predictive performance.

H Additional Experiments

Unlearn Ratio Investigation. As described in the main text, we adopt a default configuration where 5% of graph elements are selected as unlearning targets. To comprehensively evaluate MGU’s performance under different unlearning scales, we extend the unlearn ratio from 1% to 30% and report the results in Figures 12–14. Across all settings, MGU consistently achieves the highest ToU scores, demonstrating stable and reliable performance under different unlearning ratios. In contrast, baseline methods exhibit noticeably lower performance on hard tasks compared with random ones,

highlighting the intrinsic challenge of unlearning highly memorized elements. These trends are consistently observed across various datasets, GNN backbones, and unlearning types, confirming the broad generality of our findings. Overall, MGU maintains stable and superior performance, demonstrating strong robustness and adaptability to diverse graph structures and unlearning scales.

Adversarial Robustness. Figure 15 presents the robustness evaluation of GU methods against membership inference attack (MIA). From Figure 15, we can find MGU maintains consistently high and stable MIA proximity scores in both random and hard settings, while other methods exhibit notable fluctuations, especially at lower unlearning ratios in hard tasks. These results indicate that MGU effectively mitigates adversarial risks and preserves robustness under varying unlearning conditions.

I Experimental Environment

All experiments are conducted on a workstation equipped with an Intel(R) Xeon(R) 8352V CPU (2.10 GHz) and a single NVIDIA GeForce RTX 4090 GPU with 24 GB of memory. The operating system is Ubuntu 20.04.5, and the software environment includes Python 3.10, PyTorch 2.1.2, and CUDA 11.8.

J Parameter Settings

Given the diversity of GNN backbones, graph datasets, GU task types, and difficulty levels, MGU is evaluated under corresponding experimental settings to ensure fair and reliable comparisons. Due to space limitations, complete parameter settings are provided in the uploaded source files.